

Registro de las sesiones en la academia

Pasos a realizar en la academia para almacenar las sesiones que se agendan en el calendario. Documento técnico de implementación.

El calendario envía cada sesión; la academia la guarda. Una sola mutación de GraphQL para todo el ciclo de vida — alta, cancelación y reagendamiento — con un *upsert* por `reservationId`.

1 Por qué se envía en vez de leerse

Las preguntas que hay que poder responder —cuántas sesiones da cada profesor al mes, cuál es su media, qué uso real tiene el servicio— son de gestión académica, y la gestión académica vive en la academia.

La alternativa era que la academia leyera el dato del calendario y no guardarlo, para no duplicarlo. No sirve: el calendario mueve del orden de **500 agendas al día**, lleva dentro todos los embudos y en algún momento habrá que limpiar reservas viejas o la tabla no para de crecer. Un histórico académico no puede colgar de algo que está previsto borrar. Duplicar es el precio de que sobreviva a esa limpieza.

2 Qué se manda

Lo acordado y nada más: el profesor, el nombre del evento y cuándo fue. Es lo justo para auditar el uso del servicio, que es para lo que se pidió.

CAMPO	TIPO	QUÉ ES
<code>reservation_id</code>	String	Id de la reserva en el calendario. Clave del upsert.
<code>status</code>	String	<code>confirmed</code> o <code>cancelled</code> .
<code>academy_id</code>	Int?	La <code>Academy</code> . Puede llegar <code>null</code> — ver punto 6.
<code>professor_email</code>	String	En minúsculas. Empareja con <code>Professor.user</code> .
<code>student_lms_id</code>	String	El <code>UserProfile.id</code> , del token del iframe. Vacío si la reserva no vino embebida.
<code>student_email</code>	String	En minúsculas. Respaldo del anterior.
<code>event_type_name</code>	String	Ej. «Clase 1 a 1 de inglés».
<code>starts_at</code>	DateTime	ISO-8601 en UTC.
<code>ends_at</code>	DateTime	ISO-8601 en UTC. Va por el feedback (punto 8), no por el recuento.

Lo que no se manda, aunque el calendario lo tenga a mano: los nombres del profesor y del alumno —la academia ya los tiene registrados, y sus datos personales no tienen por qué viajar dos veces—; el id del tipo de evento, el teléfono, la zona horaria del alumno, la URL del Meet y la hora en que se reservó; y la duración, que se saca restando. Cada campo de más es uno que hay que mantener sincronizado entre dos aplicaciones y que alguien acabará usando para algo. Añadir uno el día que haga falta cuesta menos que quitarlo cuando ya se usa.

3 La mutación

En `api/mutations/`, con `permission_classes=[IsAuthenticatedWithAPIKey]` como el resto. Autenticación por la cabecera `X-API-Key`, la que ya valida `api/authentication.py`. El equivalente más parecido que ya existe es `syncBillingOrder`: mismo patrón de payload y misma idea de sincronizar algo que vive en otro sistema.

```
mutation SyncCalendarSession($input: SyncCalendarSessionInput!) {
  syncCalendarSession(input: $input) {
    success
    created
    errors
  }
}
```

```
@strawberry.type
class SyncCalendarSessionPayload:
    success: bool
    created: bool = False
    errors: Optional[List[str]] = None
```

Los campos del input se declaran en `snake_case`; Strawberry los expone en `camelCase` solo, igual que en `order_id` o `academy_id`.

4 El modelo

Una tabla nueva. No encaja en `CalendarEvent`: eso es el calendario propio de la academia —masterclases, recurrencias— y esto es el registro de una reserva que ya ocurrió en otro sistema. Mezclarlas obligaría a que todo lo que hoy consulta `CalendarEvent` aprenda a distinguir entre las dos.

Lo mínimo: `reservation_id` (`unique=True`), `status`, FK a `Academy`, FK nullable a `courses.Professor`, FK nullable a `accounts.UserProfile`, `event_type_name`, `starts_at` y `ends_at`.

Conviene guardar también `professor_email` y `student_email` tal como llegan, aunque las FK resuelvan: si mañana falla un emparejamiento, sin eso no hay forma de saber de quién era la sesión.

5 Upsert por `reservation_id`, siempre

Nunca insertar a ciegas. El calendario reintenta —tres veces por tarea, más un barrido periódico, más un comando de reenvío manual—, así que el mismo `reservation_id` puede llegar varias veces. Con el upsert, repetir no duplica. Igual que `sync_billing_order`: `created=True` si era nueva, `False` si actualizó una que ya estaba.

Llegan altas y bajas

- **Se agenda** → llega con `status: "confirmed"`.
- **Se cancela** → llega la misma `reservation_id` con `status: "cancelled"`.
- **Se reagenda** → llegan dos: la vieja como `cancelled` y una nueva, con otro id, como `confirmed`.

Las canceladas se guardan, pero no cuentan

Hay que almacenarlas en vez de borrarlas, y dejarlas fuera del recuento de «cuántas sesiones dio este profesor en agosto». Si se cuentan, el número mide clases que nunca ocurrieron y deja de servir para nada.

Si algo falla, `success: false`

GraphQL responde 200 aunque la mutación falle, y el error va en el cuerpo. El calendario mira el cuerpo justo para no dar por bueno un envío que no se guardó: un `success: true` de mentira es la forma silenciosa de perder el dato. Devolver el error no pierde la sesión, porque se reintenta.

6 Tres puntos donde esto se puede torcer

El profesor puede no resolverse aunque esté registrado

`Professor.user` es un `OneToOneField(null=True)`. Un `Professor` sin `user` no tiene email y no empareja con nada. Y aun teniéndolo, el email que manda el calendario es el corporativo con el que esa persona organiza allí, que no tiene por qué ser el mismo que el de su `UserProfile`.

Hay dos salidas razonables y la elección corresponde a la academia, porque cambia lo que hace el calendario: guardar la sesión con `professor` a `null` y el aviso en `errors`, o devolver `success: false` para que se reintente hasta que el profesor esté dado de alta. Con `false` el calendario reintenta y encola alerta. Lo que conviene evitar es aceptarla con `success: true` y dejarla huérfana sin rastro.

El alumno se empareja por `student_lms_id`

Ese campo es el `UserProfile.id`: el mismo `uid` que la academia ya firma dentro del token del iframe. El emparejamiento es por tanto una búsqueda por clave primaria, no por email.

El email queda de respaldo, y hace falta: **los tipos de evento que se registran no son solo los que se embeben**. La casilla que activa el envío está en todos los tipos de evento del calendario y se irá marcando lo que interese medir. En los que no se embeben nunca habrá `student_lms_id`, y en algunos ni siquiera habrá un alumno de la academia al otro lado. Esas sesiones hay que guardarlas igual: las métricas por profesor siguen valiendo, que son las que se pidieron.

`academy_id` puede venir null

La academia es multi-tenant y sus mutaciones piden `academy_id`. Desde el calendario no siempre se puede deducir: el código de escuela que usan las demás integraciones sale del embudo o del lead, y una clase 1 a 1 reservada desde la academia no tiene ninguno de los dos. Se configura a mano en cada tipo de evento.

En cuanto la academia facilite los ids de sus `Academy`, se rellenan y el campo viaja siempre puesto. Se deja opcional para que la integración no se quede parada esperando ese dato: con `null`, hay que resolverlo por el profesor.

7 Ejemplo completo

```
POST /graphql/
Content-Type: application/json
X-API-Key: <la clave>

{
  "operationName": "SyncCalendarSession",
  "query": "mutation SyncCalendarSession($input: SyncCalendarSessionInput!) { syncCalendarSession(input: $input) { success created errors } }",
  "variables": {
    "input": {
      "reservationId": "48213",
      "status": "confirmed",
      "academyId": 7,
      "professorEmail": "profesor@conquerx.com",
      "studentLmsId": "4321",
      "studentEmail": "alumna@example.com",
      "eventTypeName": "Clase 1 a 1 de inglés",
      "startsAt": "2026-09-15T14:00:00+00:00",
      "endsAt": "2026-09-15T14:45:00+00:00"
    }
  }
}
```

8 Lo que viene después: el feedback del alumno

La idea es que unos minutos después de terminar la sesión le llegue al alumno un correo pidiéndole su opinión del profesor. Eso se hace entero en la academia y no necesita nada más del calendario: con `ends_at`, el alumno y el profesor ya se puede programar el envío y colgar la puntuación de la sesión. Por eso `ends_at` va en el payload aunque para contar sesiones bastaría con `starts_at`.

9 El lado del calendario, ya hecho

- Envío asíncrono, con tres reintentos y espera creciente. Una academia caída **nunca** impide que un alumno reserve su clase.
- Un barrido periódico revisa las reservas de las últimas 24 h y reencola las que no llegaron, distinguiendo el alta de la cancelación.
- Para huecos más antiguos que eso —una caída larga— hay un comando que reenvía un rango concreto. Exige el rango a propósito: es para tapar huecos, no para volcar histórico.
- Estado por reserva visible en el admin, en una columna propia: enviada, fallida con enlace al registro del error, pendiente, o no aplica.

Configuración: `ACADEMIA_ENABLED`, `ACADEMIA_GRAPHQL_URL` y `ACADEMIA_API_KEY`. Sin ellas cada envío hace no-op y lo deja en el log, así que el calendario se puede desplegar antes de que la mutación exista.

10 Cómo se comprueba que las dos partes se entienden

1. Se monta la mutación y se crea una `APIKey` para el calendario.
2. Se configuran URL y clave en el entorno de pruebas y se marca un tipo de evento de prueba.
3. Se agenda una sesión de mentira: tiene que aparecer en la academia.
4. Se cancela: la misma fila tiene que pasar a `cancelled`.
5. Se reagenda: tienen que quedar dos filas, una cancelada y una confirmada.
6. Con eso en verde, se marcan los tipos de evento reales.

El histórico anterior no se sube

Las reservas previas a la integración se quedan donde están; el recuento arranca el día que se enciende. Si más adelante hiciera falta alguna de antes, se puede reenviar un rango con el comando, pero no es el plan.